学习清单 01 · 开源项目 × 交互评估

# ELYTerminal 可爱的终端，怎样认真验货？

从会话侧栏、点击改命令，到状态协议与透明背景。
把喜欢变成可复验的问题，把没测的留在边界里。

12 个由浅入深的疑问 | 6 步真实组件实验 | 5 种用途

**状态：学习交付已完成；原生产品体验待验证。**
真实执行了上游 Bash 集成和 OSC 133 Rust 解析器；未启动 ELYTerminal GUI，未复现 macOS 27 透明高亮问题。不是“完整终端运行通过”。

## 本项来源边界

原帖：刘佳怡 @liujiayi1111；发布时间 **2026-10-23 13:16:09（Asia/Shanghai）**，与收藏日期不是一回事。第一次接口返回截断 text，第二次显式读取 `note_tweet` 获得全文；响应未发现引用帖。三张原图已阅读，均为项目宣传图，不是本次截图。

原帖：https://x.com/i/status/2113499695586721857
上游 SHA：9504b5382f3b47b0377063fa3b78cb706f44ed08
知识库 main SHA：a8674777e18302b8804b137212b1396cbfe5a747
访问日期：2026-11-11（Asia/Shanghai）。原帖不是 Git 文件，无 commit SHA。

**延续旧仓库风格**
鲜艳青蓝橙、问题卡、大号重点、内联 SVG 和 A4 分页。参考 agent_desktop 既有指南。

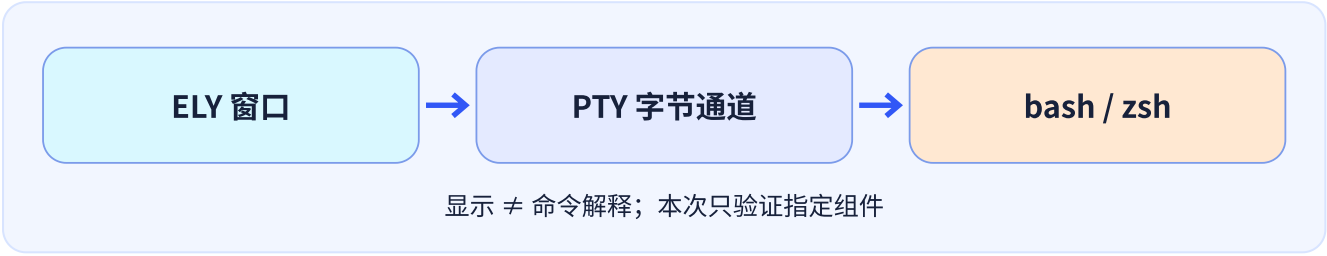
**只补本次新增知识**
读总览和四篇相关概念；vault 未检出 ELY 专项成果。既有设计令牌、状态推断、验货纪律只追加案例。

# 疑问即目录 · 入门与架构

## 01 它是一个 shell，还是终端模拟器？

**ELYTerminal 是显示与交互外壳，bash/zsh 才解释命令。** 终端模拟器把程序输出的字符和控制序列画成网格，并把键盘、鼠标输入送回程序。换主题不会让错误的命令变正确，也不是替换编译器。

当前主线以 Rust + GPUI/Ely 构建原生窗口，终端内核是 alacritty_terminal。比如 `printf` 输出颜色，shell 决定运行哪个程序，终端决定颜色怎样显示；把这三层分开，才知道卡顿或光标错误该查谁。

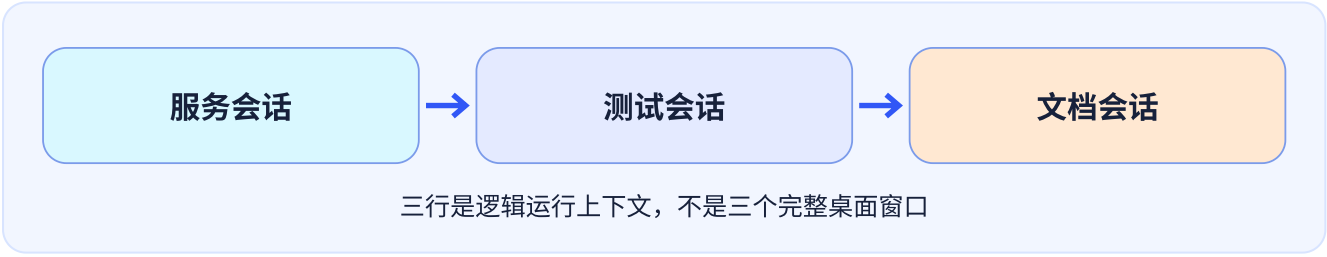



依据：README.md；Cargo.toml

## 02 会话侧栏为什么比堆窗口容易管理？

会话是一个独立终端运行上下文，不等于系统窗口。官方文档描述左侧新建、关闭、切换和拖动重排，右键复制会话会采用原 shell 与上次报告的目录；**不是把进程内存完整克隆**。

可将“服务、测试、文档”放成三行，先命名再切换，减少寻找窗口。关闭仍可能结束正在跑的进程，不能因侧栏方便就忽略代价。本次没有启动 GUI，因此“比原生终端快多少”仍须用同一任务计时，不能由作者喜爱推出效率提升。



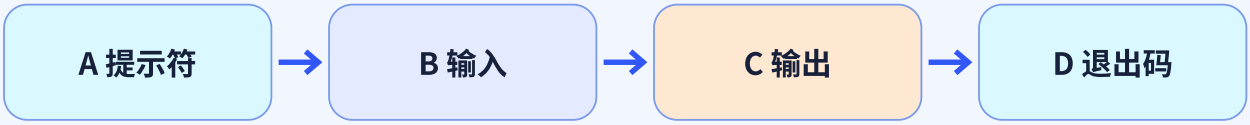

依据：docs/zh_cn/features.md:L17-L24

# 疑问即目录 · 入门与架构

## 03 PTY 和 OSC 133 怎样让窗口理解命令阶段?

PTY（伪终端）是一对供程序交互的字节端点；OSC（操作系统命令控制序列）是终端字节流中的元信息。本项目 shell 集成用 **A=提示符开始、B=输入开始、C=输出开始、D=退出状态** 标出命令边界，不靠“屏幕安静了”猜运行结束。

本次真实 Bash 子进程输出了 18 个 OSC 133 标记；`false` 后出现 `D;1`，成功命令后出现 `D;0`。目录则由 OSC 7 报告，含 % 的目录被编码为 %25。字节捕获与真实上游 Rust 解析器连接，仍不是 GPUI 完整数据通路验收。



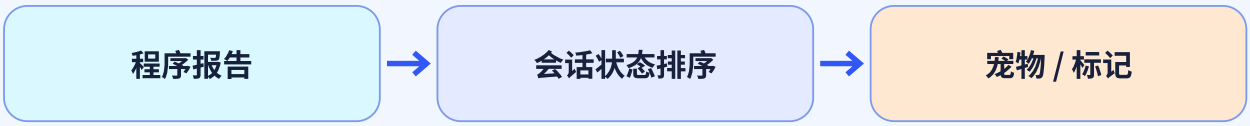

OSC 7 报目录；OSC 133 报阶段；不能混作相同信号

依据：assets/shell-integration/bash/elyterminal.bash:L25-L83；src/term/marks.rs

## 04 Ely 宠物是 AI 吗？状态可信吗?

原帖把待机打盹、错误流汗描述为有趣的体验；官方文档把宠物动作关联到等待、失败、完成和工作。这是**状态可视化**，不是一个能理解工作质量、替你保证成功的 AI 审核员。

当前 `status.rs` 的显示优先级是 blocked → error → working → done → idle，同类取最近报告；进程可用 OSC 7501 上报状态。应把上报者当作信号来源：done 不能证明文件正确。可类比已学的“可逆派生状态”，但本次没有驱动宠物动画或验证通知。




done 是被报告的状态，不是独立质量证明

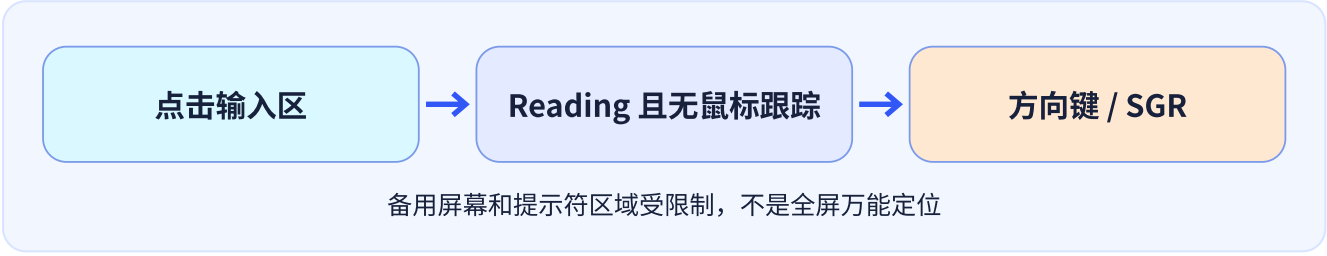
依据：docs/zh_cn/features.md:L16-L18；src/term/status.rs:L337-L353

# 疑问即目录 · 机制与边界

## 05 点击命令就能移动光标，原理是什么？

**不是直接改一块屏幕，而是把点击翻译成 shell 可接受的输入。**当前源码只在 shell 处于 Reading、程序没有跟踪鼠标时进入点击移动；默认路径按目标位置计算左右方向键数量。支持点击事件的 shell 则接收 SGR 鼠标按下序列。

例如在 `echo hello` 末尾点 h，可转换为向左 5 次。进入 vim 的备用屏幕、点到提示符或无有效输入区时会被拒绝。原帖的流畅体验有代码依据，但不等于任何程序、任意屏幕位置都能用；本次只实际验证 readline 接收方向键，未运行这个 GUI 点击函数。




依据：src/term/click.rs:L42-L92；src/term/click.rs:L154-L182

## 06 中文、宽字符和软折行为什么容易点错？

“列数”与“方向键次数”不同：中文一般占两列，但只需一次逻辑光标移动；组合字符也不能按 UTF-8 字节数计算。源码统计时跳过宽字符占位格，并把软折行接成同一输入行，还把点击过行尾的位置夹到有效终点。

上游 `click/tests.rs` 包含中文、软折行、尾空格、两行提示符和备用屏幕用例；这是**读到测试源码**，不是本次执行。下一轮 GUI 验证应用 中文ab、长命令折行及带组合音标字符，比较实际插入位置和输入内容，不可拿本次 ASCII readline 成功替代。




依据：src/term/click.rs:L55-L92；src/term/click/tests.rs:L61-L90

# 疑问即目录 · 机制与边界

## 07 为什么透明背景会让选中态“不明显”？

原帖明确把问题限定为“macOS 27、某些背景、侧栏高亮对比度”，这是作者观察，不是普遍缺陷证明。最终颜色受窗口底色、壁纸、透明度和高亮叠加共同影响；当前代码也有 glass alpha 与选中色 alpha，不能只审单个主题 hex 值。

本次用任意教学色板做 3 张纯色壁纸 × 3 档不透明度的 9 个场景。淡填充对相邻背景的比值为 **1.020013–1.298294:1**；即使加固定深色边，在深壁纸且 25% 不透明度时也只有 **2.234323:1**。所以“加深一条边就万无一失”也不成立；这些数值不是 Ely Light 或 macOS 实测。

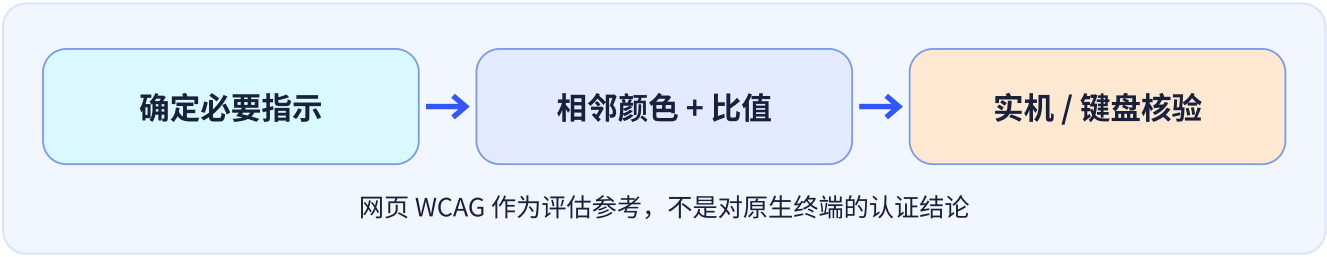



依据：src/themes/chrome.rs:L7-L50；原帖 note_tweet；自编 contrast.json

## 08 怎样把“看得清”变成可复验标准？

先明确测的是文字、焦点环还是用于识别状态的图形，再记录真正相邻的颜色。W3C 的 WCAG 2.2 非文本对比度说明，以必要状态指示与相邻颜色 **3:1** 为参考；不能把“焦点前后两个不相邻状态的色差”直接当这个标准。

练习采用 `C=αF+(1−α)B` 的通道混合，再做 sRGB 线性化：`L=.2126R+.7152G+.0722B`，比值为 `(L高+.05)/(L低+.05)`。黑白=21、同色=1 的两条端点断言通过。要做实机验收，还须在最差背景找必要标记、增加形状/文字提示，并检查键盘焦点，不能据一个颜色对宣称整应用无障碍通过。



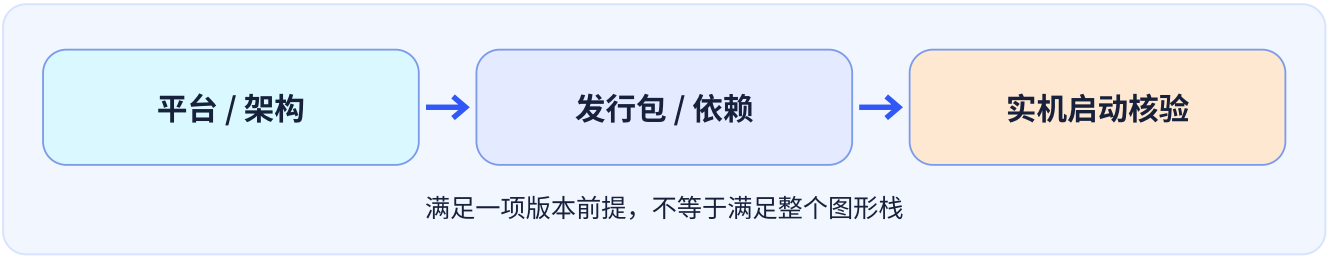

依据：W3C SC 1.4.11；exercise/lab.py contrast

# 疑问即目录 · 安装与评估

## 09 安装和系统支持究竟怎样核对?

固定主线 README 声明 macOS 11+、Apple silicon：DMG 拖入 Applications，或 Homebrew cask；签名、公证是**官方声明**，本次没有下载 macOS 安装包验证。不能据此推导 Intel Mac 支持或 macOS 27 特定视觉行为已修复。

Linux 文档要求 x86_64、glibc 2.35+、X11/Wayland 和可用图形驱动。本云端是 x86_64/glibc 2.41，但“有 DISPLAY”不证明 GPUI 能工作。完整源码要求 Rust 1.95，本环境 1.85.1，因此未做全应用构建；独立 OSC 133 模块仅依赖 log，已能用现有工具链测试。




依据：README.md:L14-L20；docs/zh_cn/install.md:L5-L39；Cargo.toml:L1-L6

## 10 可配置功能多，哪些边界值得先守住?

设置变化会立即写入平台配置文件。当前 README 说 JSON 无法解析或数值越界会阻止启动并指出字段，不会覆盖坏文件；实际排障前应备份并比较更改，不直接删除配置。这条是文档阅读结论，本次未做坏配置启动实验。

侧边 Git 面板能暂存、丢弃和提交，这些不是“只读展示”；命令行控制默认关闭，打开后可读会话、发命令。本轮不启用控制端口、不读取令牌、不使用用户 shell 配置。实验为 Bash 子进程设置空白 HOME，避免把个人启动脚本或历史误收入日志。



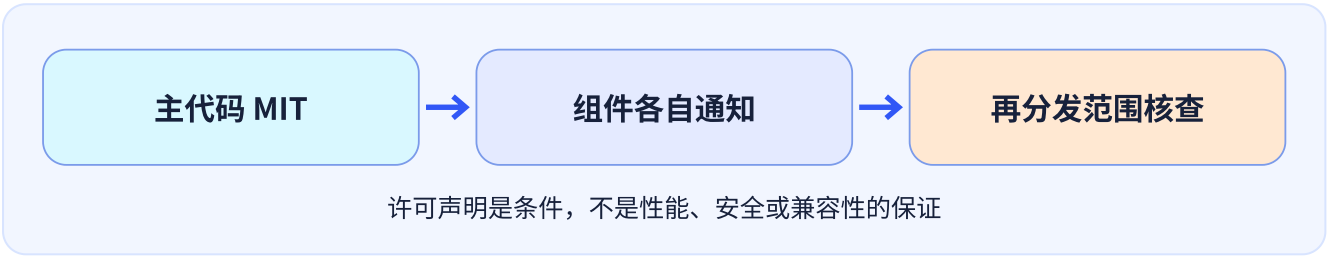

依据：README.md:L38-L46；docs/zh_cn/features.md:L24-L34；exercise/lab.py

# 疑问即目录 · 安装与评估

## 11 MIT 能说明哪些事，又不能保证什么？

本次实际读取 LICENSE：主代码 MIT，允许使用、修改与分发，但要保留版权与许可声明；原帖的 MIT 说法因此得到固定版本文本支持。练习只包含两个上游原样文件，附完整 MIT 许可，自编驱动与来源明确分开。

MIT 不保证软件质量、安全或全部依赖采用同一许可。README 另列 Ghostty、主题、SIL Open Font License 字体、X11 表与 ConPTY 通知，若整体打包仍须逐项核对。沿用已有“开源验货三查”而不新建同义概念，也不因开源就认定无需安全评估。

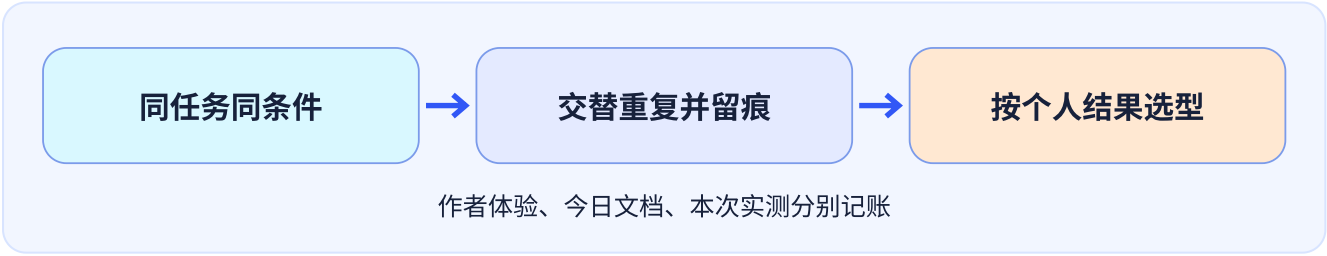



依据：LICENSE；README.md:L62-L64

## 12 如何判断适合自己，而不是被可爱宣传说服？

把“喜欢”与“任务结果”分开：同一台 Mac、同一 shell 和字体，轮换 ELYTerminal 与现用终端，做 3 会话切换、命令中部修错、长命令软折行、等待提示和最差壁纸识别。每个样例重复 5 次，记录完成时间、误操作数和是否需要猜状态；次序交替，避免熟练效应偏差。

这是**待执行的评估协议**，不是已收集数据。原帖配图标 v0.1.8，当前 main 包版本是 0.2.4，不能用今日源码证明当时体验或修复历史。本轮真实证据止于 shell/解析组件、数值模型和文档印刷；性能、内存与个人效率没有实测，官方“快约三倍”等不搬作本机成绩。




依据：docs/zh_cn/features.md:L8-L9；原帖三张配图；本次实验日志

# 它能给我什么？五个具体用途

### ① 把项目终端整理成“工作台”

让服务、测试、笔记各有会话名，快速定位和切换。真正验证的是找会话耗时、误关次数，而不是截图里有多少漂亮面板。复制会话不代表复制运行现场。

### ② 减少命令中部修错的摩擦

点击正在输入的命令定位，再键盘修改；保留备用屏幕、鼠标跟踪、中文与软折行的例外清单。Linux 本轮只验证方向键进入 readline，不保证原生点击效果。

### ③ 用余光注意等待与失败

会话标记、宠物和通知可帮助发现“程序等我了”；但最终结果仍读退出码、输出文件或业务证据，不把宠物欢呼当质量门禁。

### ④ 为自己的界面做透明度压力测试

复用九场景色板脚本，逐个检查壁纸、底色、淡高亮和边线。增加文字与形状冗余，再在实机确认，避免从孤立设计 token 推导整窗可读性。

### ⑤ 学习一个可追溯的开源拆解方法

固定 SHA → 识别两个小组件 → 真跑 PTY 和上游测试 → 把结论写回候选提案。可以用于其他终端协议学习，不碰用户密钥、生产会话或付费 API。

本页前 3 项为产品功能用途；不是本次原生体验的成功声明。后 2 项练习与方法已在本云端执行。

# 一个主实验 · 从字节到可读性

工作目录：`/workspace/learn-batch/01-elyterminal/exercise`。以下命令已实际执行，首次运行 Rust 练习需公共 crates.io 网络；不需要账户或 API Key。

## 1 锁定输入与检查最小环境

**目的：**证明拿来测试的两个文件确实来自固定上游，而不是复刻后自测。

```
python3 lab.py baseline
```

> **真实结果：**Linux x86_64 / glibc 2.41；Python 3.13.5；Bash 5.2.37；Rust/Cargo 1.85.1。两文件 SHA-256 与上游字节一致；bash -n 退出 0。

**验证方式：**看输出“2 个上游副本字节一致：PASS”“上游 Bash 语法检查：PASS”；原文件指纹留在日志。

**边界：**初稿驱动字符串转义错误曾两次退出 1，随后修复重新执行；日志未删除。

## 2 真实 PTY 跑上游 Bash 集成

**目的：**不用个人 ~/.bashrc，而在空白 HOME 中观察命令阶段、目录与方向键编辑。

```
python3 lab.py shell
```

> **真实结果：**4 场景通过：echo 成功、false 失败、含空格/% 的目录、readline 编辑。捕获 789 字节与 18 个 OSC 133 标记。

**验证方式：**查看 results/pty.raw 与 pty-readable.txt；成功 D;0、失败 D;1，目录含 %25；shell-results.json 有每场输入 hex 和输出 repr。

**边界：**脚本通过 PTY 子进程交互，不是通过 ELY 的鼠标事件；可能出现无控制终端的 Bash job-control 提示，不据此声称真实桌面运行。

# 一个主实验 · 从字节到可读性

工作目录：`/workspace/learn-batch/01-elyterminal/exercise`。以下命令已实际执行，首次运行 Rust 练习需公共 crates.io 网络；不需要账户或 API Key。

## 3 执行真实上游解析器测试

**目的：**让原样 marks.rs 的上游测试运行起来，验证 A/B/C/D、click_events 以及非法标记的既有行为。

```
CARGO_HOME=/workspace/learn-batch/01-elyterminal/_tools/cargo-home cargo test -- --nocapture
```

> **真实结果：**仅此独立练习 crate：3 passed、0 failed。依赖 log 固定为 0.4.28，第一次下载公共包，Cargo.lock 留存。

**验证方式：**输出应出现三个 marks::tests 名称；3 个测试函数不是三条断言，也不是完整 ELYTerminal 测试数量。

**边界：**这个 crate 使用 Rust 1.85.1 可编译，不说明整个 Rust 1.95 项目可用。

## 4 把真实捕获交给真实解析器

**目的：**连起上游 Bash 产生的标记与上游 Rust 解析规则，而非把预期输出硬写进结果。

```
CARGO_HOME=/workspace/learn-batch/01-elyterminal/_tools/cargo-home cargo run -- results/bodies.txt
```

> **真实结果：**18 个 bodies 逐行解析，A;aid=elyterminal → Prompt(Arrows)，B → Input，C → Output，D;1 → Done(Some(1))。

**验证方式：**打开 results/bodies.txt 对照逐行 stdout；记录中保留 named 未使用的编译警告，不把警告藏掉。

**边界：**这里只传递提取的 OSC 133 body，没有运行 ELY tap、alacritty 网格或 GPUI 渲染。

# 一个主实验 · 从字节到可读性

工作目录：`/workspace/learn-batch/01-elyterminal/exercise`。以下命令已实际执行，首次运行 Rust 练习需公共 crates.io 网络；不需要账户或 API Key。

## 5 做九场景透明度数值对照

**目的：**用可控的 sRGB 简化模型检查“淡高亮在不同背景下是否稳定”；生成能看的中间产物。

```
python3 lab.py contrast
```

> **真实结果：**9 个场景；淡填充 1.020013–1.298294:1，固定深边最差 2.234323:1。黑白=21、同色=1 两条数学断言通过。

**验证方式：**查看 results/contrast.json 与 contrast-demo.html。全部淡填充低于自选参考 3:1，不使用四舍五入值判断阈值。

**边界：**这是教学色板，不是 Ely Light 色值或 macOS 合成器；模糊、色彩管理和真实壁纸待实机核验。

## 6 记录完整应用与平台边界

**目的：**核对实际构建前提，不让局部组件成功冒充原生终端成功。

```
python3 lab.py inventory
```

> **真实结果：**cargo metadata --offline --no-deps 成功读取 package=0.2.4、rust_version=1.95；环境 1.85.1，不执行全应用构建。

**验证方式：**对照上游 Cargo.toml 与真实 metadata 结果；明确没有 cargo test 全量报告、GUI 截图或性能数据。

**边界：**macOS 11+ Apple silicon 安装、macOS 27 透明问题、多会话切换/关闭、宠物、中文与折行点击均待验证；未测试 Windows。

# 证据账本与下一步

| 证据层 | 本轮结论 | 不能外推 |
| --- | --- | --- |
| 原帖全文 / 三图 | 主观体验与 MIT、功能线索均读到；无引用帖字段 | 截图宣传不等于实机复现 |
| 上游固定快照 | 安装/许可/点击边界/状态优先级有文档和源码定位 | 今日 main 不等于收藏时 v0.1.8 |
| 真实组件执行 | Bash 四场景、Rust 上游三测试、18 标记解析 | 不是全量构建 / 原生终端验收 |
| 数值模型 | 九个合成透明度场景与两个端点断言 | 不是 macOS 27 bug 复现或修复证明 |
| 自查 | 指南印刷和逐页视觉检查另见 pdf-review.md | 没有独立审核，不伪造审核人 |

## macOS 人工验收卡（全部待验证）

1. 记录 Mac 架构、系统/应用版本，按官方 README 安装；核对签名/公证，不绕过安全防护。
2. 新建 3 个会话分别运行无副作用命令，命名、切换和关闭空闲会话；不关闭重要任务。
3. 输入但不执行一条测试命令；在 ASCII、中文、折行和组合字符处点击编辑，记录光标和最终字符串。
4. 用已知成功/失败命令比较退出码、会话标记和宠物；不把程序自报完成当任务真完成。
5. 保持 Ely Light，变换明暗/高频壁纸与不透明度，记录选中态和键盘焦点是否仍可辨；比较原生终端的相同任务。

## 一手来源（完整清单见 sources.md）

原帖：x.com/i/status/2113499695586721857；全文与配图可读；日期 2026-10-23 13:16:09（Asia/Shanghai）。
上游：ELY-Open-Source/ELYTerminal @ 9504b5382f3b47b0377063fa3b78cb706f44ed08；README / LICENSE / Cargo.toml、中文安装和功能文档、marks.rs / click.rs / status.rs / chrome.rs。
知识库：Smashwinny/reminder_knowlege @ a8674777e18302b8804b137212b1396cbfe5a747；00-总览与四篇已有概念，agent_desktop 旧指南样式。
W3C：WCAG 2.2 SC 1.4.11 非文本对比度说明；官方网页，无本轮 Git SHA，已留存页面 SHA-256。全部访问日期为 2026-11-11（Asia/Shanghai）。

> **交付不等于任务勾选：**本轮只生成隔离候选文件，不改 vault、不提交/发布、不新建长期队列。每一条额外产品结论都应另补真实证据。